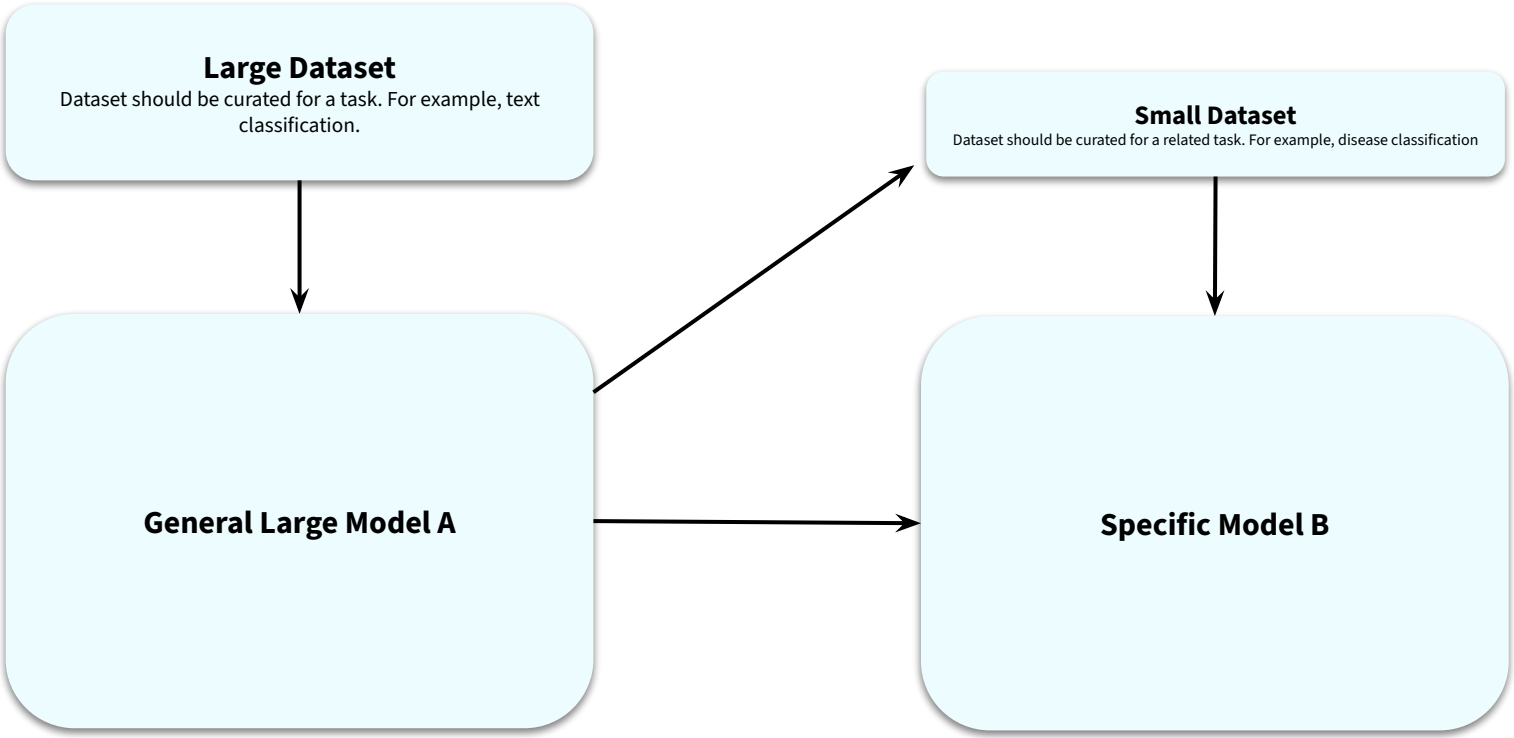
GeeksforGeeks

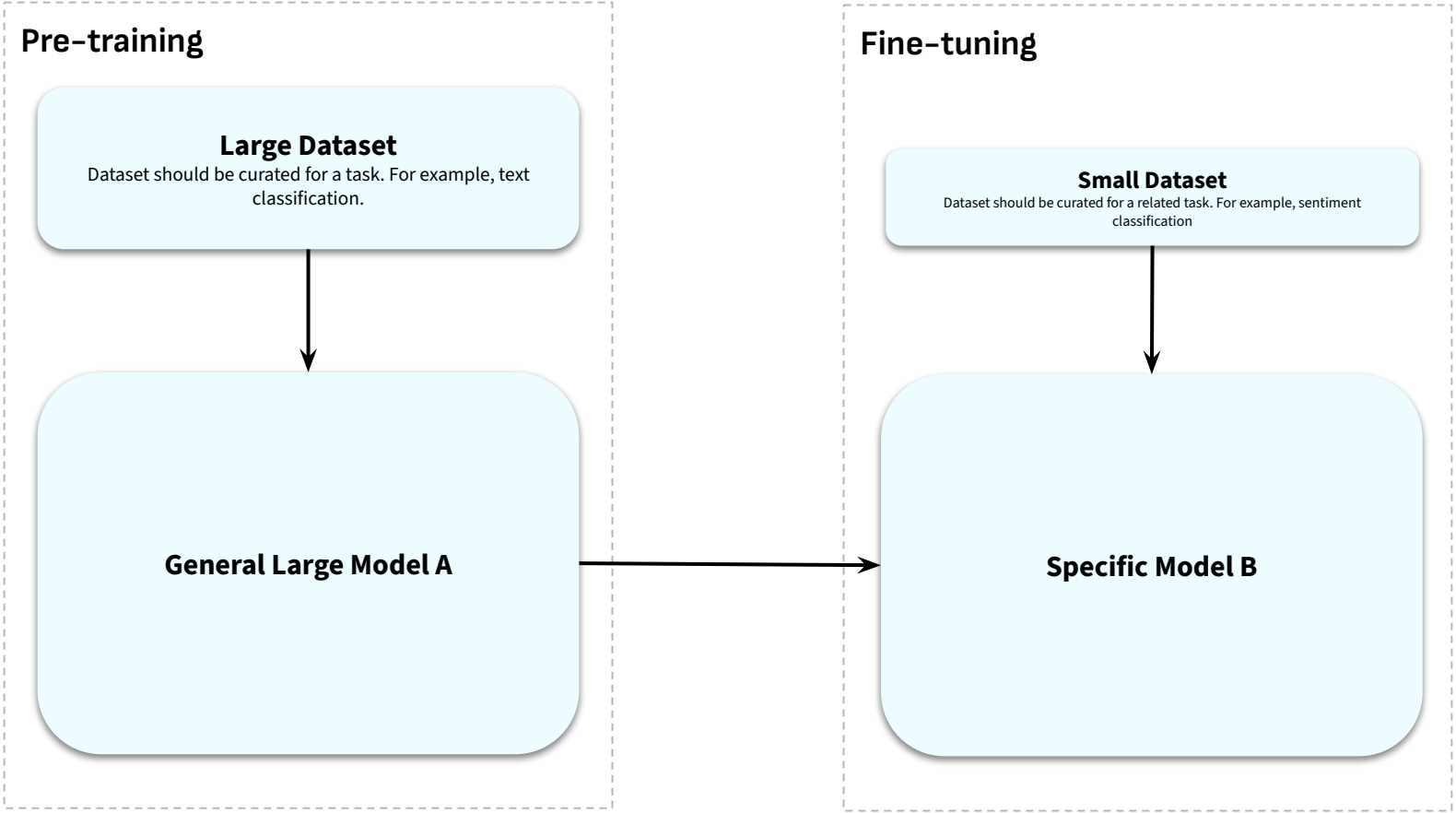
What is Transfer Learning?

Transfer Learning with Hugging Face

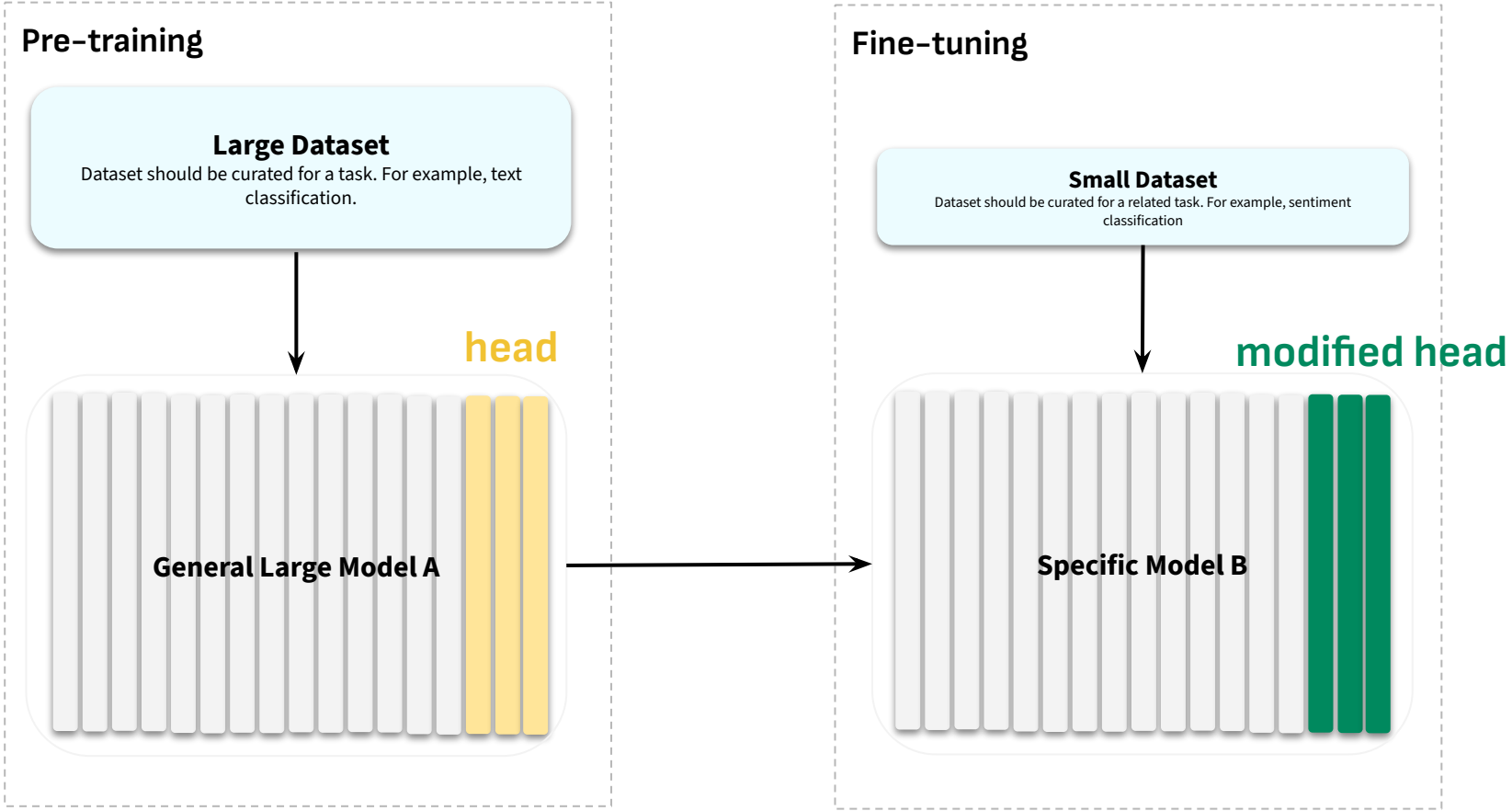
Transfer Learning



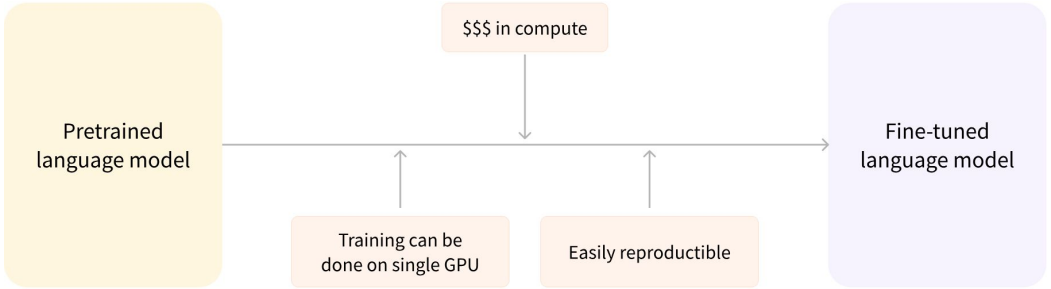
Pre-training vs Fine-tuning



Why is it called fine-tuning?



The Advantages of Transfer Learning



Source: Hugging Face

Reduced Training Time & Cost

Improved Performance

Data Efficiency

GeeksforGeeks

The TrainingArguments Class

Transfer Learning with Hugging Face

Parameter	Type	Description & Purpose
output_dir	str	**(Required)** The directory where all outputs—including model checkpoints, configuration files, and training logs—will be saved.
num_train_epochs	float	The total number of times the training loop will iterate over the entire training dataset.
per_device_train_batch_size	int	The number of training examples to process on a single device (GPU/CPU) in one forward/backward pass.
per_device_eval_batch_size	int	The number of evaluation examples to process on a single device during an evaluation run.
push_to_hub	bool	If True, the final trained model will be automatically uploaded to your Hugging Face Hub account, facilitating sharing and collaboration.

Parameter	Type	Description & Purpose
learning_rate	float	The initial step size for the optimizer. This is one of the most crucial hyperparameters to tune for successful training. A typical value for fine-tuning is between 2e-5 and 5e-5.
weight_decay	float	A regularization technique that penalizes large weights in the model to prevent overfitting. It adds a term to the loss function that encourages smaller weight values. A common value is 0.01.
gradient_accumulation_steps	int	A powerful technique to simulate a larger batch size without increasing memory usage. The trainer will execute this many "mini-batches" and accumulate their gradients before performing a single optimizer step. Effective Batch Size = per_device_train_batch_size * num_gpus * gradient_accumulation_steps.
eval_strategy	str	Defines when to run evaluation during training. Can be "no", "steps" (evaluate every eval_steps), or "epoch" (evaluate at the end of each epoch).
save_strategy	str	Defines when to save a model checkpoint. Has the same options as eval_strategy.
logging_strategy	str	Defines when to log training metrics. Has the same options as eval_strategy.

Parameter	Type	Description & Purpose
load_best_model_at_end	bool	When set to True, the Trainer will keep track of the best model (based on the specified metric, e.g., validation loss or accuracy) during training and will load that model's weights at the end of the train() method. Requires evaluation_strategy to be set.
metric_for_best_model	str	The metric to use to compare models and identify the best one. For example, "accuracy" or "f1". Defaults to "loss" if not specified.
fp16	bool	If True, enables 16-bit mixed-precision training on compatible NVIDIA GPUs. This can significantly speed up training (up to 2x) and reduce memory consumption, allowing for larger batch sizes.

GeeksforGeeks

The Trainer API

Transfer Learning with Hugging Face

Without the Trainer API, you will need to write code to...

Iterate over datasets using a DataLoader.

Move data batches to the correct device (GPU/CPU).

Perform the forward pass.

Calculate the loss.

Perform the backward pass to compute gradients.

Update the model's weights.

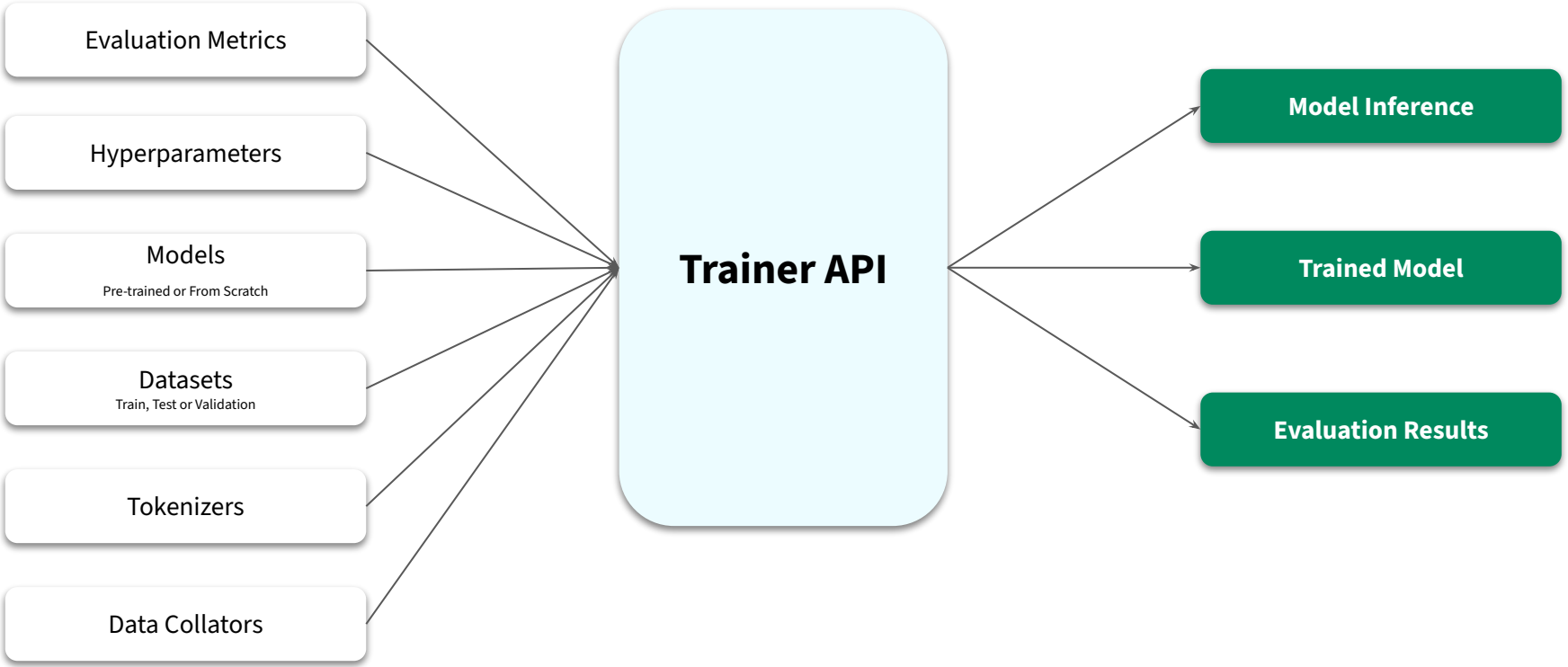
Manage the learning rate.

Run evaluation loops.

Implement logging, checkpointing, and progress bars.

Manage distributed training across multiple GPUs.

Trainer API


Evaluation Metrics
Hyperparameters
Models
Pre-trained or From Scratch
Datasets
Train, Test or Validation
Tokenizers
Data Collators
Trainer API
Model Inference
Trained Model
Evaluation Results